COMP 431 — INTERNET SERVICES & PROTOCOLS

Kevin Jeffay

Spring 2026

Homework 4, February 12

Due: 12:30PM, February 24

Building an SMTP Client/Server System Using Sockets

In this assignment you will write two programs. The first will extend and convert your SMTP "server" program (HW 2) into a "real" SMTP server. The second will extend and convert your SMTP "client" program (HW 3) into a real user mail agent and SMTP client. And as real clients and servers, your HW 4 programs will interoperate over a network by using a TCP socket.

To do this you will need to:

- Implement the final portions of the SMTP protocol: the generation of the "greeting" message by the server upon receipt of a connection, and the generation of the HELO message by the client in response to the server's greeting message
- Replace your `stdin`/`stdout` I/O for SMTP messages with socket I/O, and
- Do some additional message processing and formatting in your SMTP client.

SMTP Server Program

For I/O, your SMTP server program will now create a listening socket on a specified port number and create connection sockets for each client that connects to it. Your SMTP server program will require a single command line argument: the port number on which it should listen for connections from clients.

When your server accepts a connection from a client, in accordance with the SMTP protocol, it should generate and send the SMTP server "greeting" message (remember that SMTP is a unique protocol in that the server "speaks" first). The format of the greeting message is:

```
220 Random "greeting" text that includes the name of the server
```

The "220" return code is required; the text of the greeting message is arbitrary, however, it is traditional for the greeting text to specify the hostname (or at least the domain) of the server. For this assignment your server program should generate the greeting message:

```
220 hostname.cs.unc.edu
```

where "hostname" is the name of the machine that is running your server (*i.e.*, most likely the class server *comp431-1sp26.cs.unc.edu*). After sending this message the server must receive and acknowledge a valid SMTP HELO message from the client before proceeding with mail processing (see the following section). The HELO message is acknowledged by the server with a 250 message. By convention, the text of the acknowledgement of the HELO message echoes the text of the HELO message and includes the phrase "pleased to meet you."

For example, a sample set of "traditional" messages sent at the start of an SMTP session between an SMTP server running on *comp431-1sp26.cs.unc.edu* and a client running on *comp431-2sp26.cs.unc.edu* might look like the following:

```
220 comp431-1sp26.cs.unc.edu
HELO comp431-2sp26.cs.unc.edu
250 Hello comp431-sp26.cs.unc.edu pleased to meet you
```

The formal grammar for the HELO message is given below. Once the HELO message has been received and acknowledged, the server enters its email message processing loop and begins processing SMTP messages until the QUIT message is received.

After receiving a QUIT message, the server should send the following response to the client:

```
221 <server hostname> closing connection
```

where "`<server hostname>`" is just the name of the host on which the server is running (printed without the angle brackets). After emitting this response, the server should close its socket to the client and return to its initial state and await the next connection from a next client.

As in Homework 2, when the server processes mail messages, it should append each message that it receives into a "forward-file" (in the subdirectory "*forward*" which you can assume already exists). However, there are two differences in how the forward file will be created. First, as explained in more detail below, the "From:" and "To:" email message headers that are written to the forward file for each email message will come from headers in the received email message and not from the MAIL FROM and RCPT TO commands. That is, the "From:" and "To:" email message headers will come from the email message data sent as part of the DATA command as explained below.

Second, different from Homework 2, the filename of the forward-file should be the domain name of the receiver as specified in the RCPT TO field of the SMTP message. Thus, for example, if a message is received by the server addressed to *jeffay@cs.unc.edu* (*i.e.,* the server received the message "RCPT TO: <jeffay@cs.unc.edu>"), then the message should be appended to the file *forward/cs.unc.edu* in the current working directory.

If a message is being sent to multiple users and two or more of the recipients are in the same domain (*i.e.*, the server receives two or more RCPT TO messages that specify a user in the same domain), then only one copy of the message should be appended to the forward file for that domain. If a message is being sent to two (or more) users in different domains then the server appends a copy of the received message into the forward file for each of the different domains. Thus, for example, if your server receives the following messages during the transmission of an email message:

```
RCPT TO: <jeffay@cs.unc.edu>
RCPT TO: <user1@cs.unc.edu>
RCPT TO: <user2@unc.edu>
```

one copy of the email would be appended to the forward-file *forward/cs.unc.edu* and one copy would appended to the file *forward/unc.edu*.

Your server should deal with any protocol errors it encounters as your HW2 program did. That is, if your server encounters a protocol error in some message it receives from the client, your server program should generate the appropriate 500, 501, or 503 response message. However, now your server should send the error response message to the client via the connection socket and not print the message to standard output. If your server encounters any non-protocol error from which it cannot recover, the server

should close its connection to the client (if a connection existed), print to standard output a meaningful 1-line error message, and return to its initial state and await the next connection from a new client. If an error occurs when creating the welcoming socket, simply print a meaningful 1-line error message to standard output and terminate your program.

Your server program should perform *all* of its protocol I/O to a socket. In particular, it should not interact with the user directly in any way. Specifically, it should not output prompts or other messages (other than non-protocol error messages) and it should not read commands or other inputs from the keyboard or a file.

The server program, like most servers, will conceptually never terminate. It will be terminated by some external means such as typing *control-C* in the shell.

User Mail Agent/SMTP Client Program

Your second program will be both a simple user mail agent and an SMTP client. This mail agent program will interactively allow a user to compose and "send" an email message using a command line interface (just like email clients from the 1980s!).

Your mail agent should take *two* command line arguments: a hostname followed by a port number. The hostname is the name of the host where the STMP server that the client program will use is running (*e.g.*, *comp431-1sp26.cs.unc.edu*). The port number specifies the port number on which your SMTP server is listening for connections. (As explained below, for this assignment you will *not* be using the reserved port numbers for SMTP.) This port number should be the same as the corresponding port number parameter given to your server program.

Your mail agent should generate text prompt messages to standard output to prompt the user to manually enter in the contents of an email message. There should be a prompt message (terminated with a newline) for each of the message's main components generated in the following order:

From:
To:
Subject:
Message:

The From/To prompts will be used to get the sender/recipient information from the user. Users should be able to specify multiple email recipients by providing a list of comma-separated email addresses (with optional whitespace after the comma) on the To: line.

For example, if a user used your mail agent to compose a message, the following outputs and inputs would be seen in your terminal window (the bold text indicates outputs generated by your program and the non-bold text indicates inputs typed in by the user):

From:
jeffay@cs.unc.edu
To:
ythant@unc.edu
Subject:
How the heck are we going to grade HW4?!
Message:
Ye Mon – How are we going to grade HW4? It's going to require executing two programs for each student on two separate machines and this is going to be difficult to automate!
.

Your program should confirm that data entered in response to both the From and To prompts conforms to the syntax of the `<forward-path>` and `<reverse-path>` non-terminals in the SMTP grammar (but *without* the angle brackets) for the SMTP MAIL FROM and RCPT TO messages. If the data entered is erroneous, your client should print out a 1-line English (*i.e.*, not an SMTP) error message identifying the error (similar to what you did in Homework 1) and re-prompt the user for the correct input.

Assume users will terminate their message text by entering in a period on an otherwise blank line. (Note the period on the blank line on the last line in the example above.)

After the user has entered in a valid email message, your mail agent will create a TCP socket to the SMTP server at the host and port number specified on the command line, and forward the user's message to the server using the SMTP protocol. To do this, there are four new operations that must be added to the SMTP client portion of your mail agent program. First, when your program now connects to the server it must be prepared to receive a correct server greeting message. (Your program will do nothing with the greeting message other than receive it and confirm that it is a valid greeting message.) If the greeting message is not a valid greeting message you should print a 1-line error message to standard output and then terminate the program. If the greeting is valid, your program should reply to the greeting with the SMTP HELO message whose format is given by the grammar:

```
HELO <whitespace> <domain> <nullspace> <CRLF>
```

where `domain` is simply the domain name of the machine on which you are running your mail agent program (*e.g.*, *cs.unc.edu*). The syntax for the domain name should be consistent with the non-terminal `<domain>` described in Homework 1. (Note that many SMTP clients use a hostname rather than a domain name in the HELO message but technically the protocol specifies the use of a domain name and syntactically a host name is the same as a domain name.)

Second, when your client sends the user's email message to the server (via the SMTP DATA message), your client should format the message as follows (and as described in RFC 822):

> From: *reverse-path (with angle brackets)*
> To: *comma separated list of one or more forward paths (each with angle brackets)*
> Subject: *Text of the subject line formatted exactly as typed in by the user*
>
> *Text of the message formatted exactly as typed in by the user.*

In the case that multiple email recipients were specified by the user, a comma separated list of email addresses (with optional whitespace after the comma) should be given on the To: line. Notice that in this message format, the email headers are separated from the body of the email message by a blank line. This blank line is required by RFC 822. Note also that the format for the message and the headers here are different from the SMTP protocol messages.

For example, if I wanted to send the LA and Prof. Ketan Mayer-Patel a one-line email message with the text "This is a test," then ultimately, your mail agent would send the following SMTP messages and data (lines of email message text) to the SMTP server (after the SMTP "HELO" handshaking):

```
MAIL FROM: <jeffay@cs.unc.edu>
RCPT TO: <kmp@cs.unc.edu>
RCPT TO: <ythant@unc.edu>
DATA
From: <jeffay@cs.unc.edu>
To: <ythant@unc.edu>, <kmp@unc.edu>
Subject: Test message

This is a test.
.
QUIT
```

At the server, the contents of the DATA message (*i.e.*, the email message) should be written/appended to the forward file. Note that unlike HW 2, the "From:" and "To:" email message headers in the forward file will now be coming from the contents of the DATA message and not from the MAIL FROM and RCPT TO messages. However, the net effect of this is that format of the forward file will be essentially the same as it was in HW 2 and a line starting with "From:" will be assumed to indicate the start of a new email message. For example, if your server received the SMTP messages above, the following would be the contents of the file *forward/cs.unc.edu* and *forward/unc.edu* on the server:

```
From: <jeffay@cs.unc.edu>
To: <kmp@cs.unc.edu>, <ythant@unc.edu>
Subject: Test message

This is a test.
```

(where the text "This is a test." is the last line in the forward file). Note that as is HW2, the period on an otherwise blank line that terminates the DATA message is *not* written to the forward file.

Third, your mail agent program will terminate when either end-of-file is reached on standard input, the outgoing mail message has been successfully sent to the server, or when any SMTP or socket error is encountered (*e.g.*, errors opening the socket, errors reading from or writing to the socket, SMTP protocol errors encountered, *etc*.). If your client encounters an error it should close its connection to the server and print to standard output a meaningful 1-line error message before terminating.

Lastly, notice that when your client emits the QUIT message, the client now has to be prepared to receive the server's final 221 "connection closed" response. (This means your client can't terminate after sending the QUIT message.)

In total, the dialog between your client and server programs should be as shown in the SMTP examples shown in the textbook and lecture notes (although the text of the response messages is allowed to vary).

Notes on Picking a Port Number For Your Socket

For this assignment the client and server programs will execute on different computers. You should run your server on *comp431-1sp26.cs.unc.edu* and your mail agent/client on *comp431-2sp26.cs.unc.edu*. The port number you should use for your server is 8000 + the last 4 digits of your PID (that is, numerically add the number 8000 to the number formed by the last 4 digits of your PID). This will minimize the possibilities of a port number conflict between two students trying to run their server on the same computer using the same port number. However, it will not guarantee that port conflicts do not occur! Thus, your program *must* be prepared to deal with errors that occur when trying to create a socket on a

port number that is in use by someone else (and generally deal with any errors that occur on any socket operation).[1]

Grading

As per the instructions distributed with Homework 1, create a subdirectory named HW4 in your home directory on the *comp431-1sp26.cs.unc.edu* server, and place the final version of your program in this directory before the due date. You should have two Python files in your HW4 directory: one program should be named *Client.py* and the other should be named *Server.py*. To make it easy to grade your programs please include any "helper" classes you write inside of your Client and Server programs so that we only have to execute these two programs to test your code. As with prior assignments, email the LA when your program is ready for grading.

As with previous assignments, your programs should be neatly formatted (*i.e.*, easy to read) and well documented.

In order for your client program to be correct, it should be the case that the mail message sent to your server is compliant with the header/body format specified in RFC 822 (see the lecture notes).

Extra Credit — MIME Processing

For extra credit, in addition to your Client.py program, write a *second* user mail agent/SMTP client program (name this program *ClientEC.py*) that will allow a user to attach a file to their email. In addition, you will make your second email user agent work with the Department's SMTP server *smtp.cs.unc.edu*. You should not attempt any aspect of the extra credit portion of this assignment until you're *certain* that your *Client.py* and *Server.py* programs are *completely* correct (*i.e.*, your program has satisfied all the requirements stated above).

After reading in an email message from the user, your second mail-agent program will prompt the user for the name of a file (a Linux file path) to attach to the email. Thus, the sequence of prompts your extra credit client will generate for the user are:

```
From:
To:
Subject:
Message:
Attachment:
```

You may assume that when prompted for an attachment, the user will specify a single valid file name. Once your client has the message contents and attachment file name, it will compose a multipart MIME email message containing the text of the message as the first part and the contents of the attachment MIME encoded as a binary file as the second part. (See the textbook, the lecture notes, or RFC 2045 and 2056 for the proper MIME formats.) Your SMTP client will then send the MIME encoded email message to an SMTP server, starting with your SMTP server program (for testing purposes).

---

[1] Note that a common reason for a port number conflict is that when debugging your server program, the program terminates abnormally and does not close the socket it may have created. In this case, a subsequent execution of your server program will run into a port number conflict with the previous execution of the same program. In this case, the conflict an only be remedied by "killing" (via the Linux *kill* command) the previous instance of your program.

Once you have your *ClientEC* program working correctly, test it further by using your new mail agent program to send a MIME encoded email message to yourself (and only yourself) at your *unc.edu* email address (and only this address) by using the Department's SMTP server *smtp.cs.unc.edu* (*i.e.*, by running your *ClientEC* program with the command line arguments "smtp.cs.unc.edu 25"). Note that in order to use the Departmental SMTP server your mail agent program must be running on a Departmental server such as *comp431-1sp26*. You will not to be able to connect to our SMTP server directly from your laptop.

If your program is working correctly you should be able read the message you sent using a standard POP/IMAP client (*i.e.*, using whatever program you currently use for reading your email). The message should appear as a text message with a file attachment.

REALLY IMPORTANT NOTE: You can only send messages to either your HW 4 server program or *smtp.cs.unc.edu* and these messages must be sent from your *unc.edu* email address and sent only to your *unc.edu* email address. In particular, sending email to anyone other than yourself, the instructor, or the LA, or sending email that appears to be from someone other than yourself will be considered an Honor Code violation. Both of the class servers, as well as *smtp.cs.unc.edu*, generate and maintain extensive logs. If you attempt to misuse any of these servers by, for example, sending email to anyone other than yourself, this activity will be logged and flagged.

When you're *certain* your MIME client program is working correctly, send a single multipart email message to the LA and instructor (using our *cs.unc.edu* email addresses) that contains some polite text with a jpeg attachment of your favorite or most goofy image of you (or your pet if you're shy). This email has to be sent as a single message with multiple recipients listed on the "To:" field, and the email must contain a Subject: line.

REALLY REALLY IMPORTANT NOTE: To get extra credit for this assignment, you have ONLY ONE chance to send your multipart-MIME email message to the instructor and LA. Therefore, don't send us an email message until you are absolutely, 100% certain that your program works! If you send us multiple email messages only the first one received will be used to determine if you get extra credit.

Also, to be eligible for extra credit, this assignment must be submitted on-time. That is, if you submit the assignment after the due date and time, you CANNOT receive extra credit.